

Rapport d'analyse : Génération et visualisation de distributions statistiques

PINCAUD Steve, SAVIARD Matthieu et Henri Charonnet

29 avril 2026

Résumé

Ce rapport présente une implémentation en Python pour générer et visualiser les distributions statistiques suivantes : uniforme entière, uniforme réelle, normale, exponentielle et Poisson . L'objectif est d'analyser le comportement de ces distributions en fonction de la taille de l'échantillon, à l'aide d'histogrammes.

Table des matières

1 Introduction

Ce projet vise à modéliser et visualiser des distributions statistiques fondamentales en utilisant les bibliothèques `numpy` et `matplotlib`. L'implémentation repose sur une architecture orientée objet, avec une classe de base `Distribution` et des sous-classes pour chaque type de distribution. Les histogrammes générés permettent d'observer la convergence des échantillons vers les distributions théoriques lorsque la taille de l'échantillon augmente.

2 Prérequis

Pour exécuter ce code, les éléments suivants sont nécessaires :

- **Python 3.8+** : Langage de programmation utilisé.
- **Bibliothèques** :
 - `numpy` : Pour la génération de nombres aléatoires et les opérations numériques.
 - `matplotlib` : Pour la visualisation des histogrammes.
- **Environnement** : Le code peut être exécuté dans n'importe quel environnement Python (Jupyter Notebook, IDE comme PyCharm ou VS-Code, ou en ligne de commande).

3 Architecture du code

3.1 Classe de base : Distribution

La classe `Distribution` est une classe abstraite qui définit l'interface commune à toutes les distributions. Elle contient :

- Un attribut `name` pour identifier la distribution.
- Un générateur aléatoire `rng` de type `numpy.random.Generator`.
- Une méthode abstraite `generate(n)` qui doit être implémentée par chaque sous-classe pour générer un échantillon de taille `n`.

3.2 Sous-classes de distributions

Chaque sous-classe hérite de `Distribution` et implémente la méthode `generate(n)` selon la distribution spécifique :

- `UniformeEntiere` : Distribution uniforme discrète sur l'intervalle `[low, high]`. Utilise `rng.integers`.
- `UniformeReelle` : Distribution uniforme continue sur l'intervalle `[low, high)`. Utilise `rng.uniform`.
- `Normale` : Distribution normale (gaussienne) avec moyenne `mu` et écart-type `sigma`. Utilise `rng.normal`.
- `Exponentielle` : Distribution exponentielle avec paramètre d'échelle `scale`. Utilise `rng.exponential`.
- `Poisson` : Distribution de Poisson avec paramètre `lam` (). Utilise `rng.poisson`.

3.3 Classe HistogrammeVisualiser

Cette classe permet de générer et sauvegarder les histogrammes pour une liste de distributions et une liste de tailles d'échantillons `ns`. Elle propose une méthode `tracer()` qui :

- Crée une figure avec un sous-graphe par taille d'échantillon.
- Génère un histogramme pour chaque distribution et chaque taille d'échantillon.
- Sauvegarde les figures au format PNG (optionnel).
- Affiche les figures à l'écran (optionnel).

4 Description du code

Voici le code complet avec des commentaires détaillés :

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 class Distribution:
5     """Classe de base repr sentant une distribution statistique."""
6     "
7
8     def __init__(self, name: str, rng: np.random.Generator):
```

```

9     self.name = name
10    self.rng = rng
11
12    def generate(self, n: int) -> np.ndarray:
13        raise NotImplementedError("La m thode generate() doit tre
14        impl ment e dans la sous-classe.")
15
16    def __repr__(self):
17        return f"{self.__class__.__name__}(name={self.name!r})"
18
19    class UniformeEntiere(Distribution):
20        """Distribution uniforme discr te sur [low, high]."""
21
22        def __init__(self, rng, low: int = 20, high: int = 40):
23            super().__init__(f"Uniforme enti re ({low} {high})", rng)
24            self.low = low
25            self.high = high
26
27        def generate(self, n: int) -> np.ndarray:
28            return self.rng.integers(self.low, self.high + 1, size=n)
29
30    class UniformeReelle(Distribution):
31        """Distribution uniforme continue sur [low, high)."""
32
33        def __init__(self, rng, low: float = 2.0, high: float = 3.0):
34            super().__init__(f"Uniforme r elle ({low} {high})", rng)
35            self.low = low
36            self.high = high
37
38        def generate(self, n: int) -> np.ndarray:
39            return self.rng.uniform(self.low, self.high, size=n)
40
41    class Normale(Distribution):
42        """Distribution normale (gaussienne)."""
43
44        def __init__(self, rng, mu: float = 0, sigma: float = 0.5):
45            super().__init__(f"Normale ( ={mu}, ={sigma})", rng)
46            self.mu = mu
47            self.sigma = sigma
48
49        def generate(self, n: int) -> np.ndarray:
50            return self.rng.normal(self.mu, self.sigma, size=n)
51
52    class Exponentielle(Distribution):
53        """Distribution exponentielle."""
54
55        def __init__(self, rng, scale: float = 4):
56            super().__init__(f"Exponentielle (moy={scale})", rng)
57            self.scale = scale

```

```

65
66 def generate(self, n: int) -> np.ndarray:
67     return self.rng.exponential(self.scale, size=n)
68 '''
69
70 class Poisson(Distribution):
71     """Distribution de Poisson."""
72
73     '''
74     def __init__(self, rng, lam: float = 1):
75         super().__init__(f"Poisson (={lam})", rng)
76         self.lam = lam
77
78     def generate(self, n: int) -> np.ndarray:
79         return self.rng.poisson(self.lam, size=n)
80     '''
81
82 class HistogrammeVisualiser:
83     """Gère et sauvegarde les histogrammes pour une liste de
84     distributions."""
85
86     '''
87     def __init__(self, distributions: list[Distribution], ns: list[int
88     ]):
89         self.distributions = distributions
90         self.ns = ns
91
92     def tracer(self, save: bool = True, show: bool = True):
93         for dist in self.distributions:
94             fig, axes = plt.subplots(1, len(self.ns), figsize=(16, 3))
95             fig.suptitle(dist.name, fontsize=13)
96
97             for ax, n in zip(axes, self.ns):
98                 data = dist.generate(n)
99                 ax.hist(data, bins='auto', edgecolor='white', linewidth
100                 =0.4)
101
102                 ax.set_title(f"n = {n}")
103                 ax.set_xlabel("Valeur")
104                 ax.set_ylabel("Fréquence")
105
106             plt.tight_layout()
107
108             if save:
109                 filename = f"hist_{dist.name[:10].strip()}.png"
110                 plt.savefig(filename, dpi=150)
111
112             if show:
113                 plt.show()
114
115             plt.close(fig)
116     '''
117
118     if __name__ == "__main__":
119         rng = np.random.default_rng(seed=3)
120
121         '''
122         distributions = [

```

```

119     UniformeEntiere(rng),
120     UniformeReelle(rng),
121     Normale(rng),
122     Exponentielle(rng),
123     Poisson(rng),
124 ]
125
126 ns = [10, 100, 1000, 10000]
127
128 visualiseur = HistogrammeVisualiser(distributions, ns)
129 visualiseur.tracer(save=True, show=True)
130 '''

```

Listing 1 – Implémentation des distributions statistiques

5 Analyse des résultats

5.1 Comportement des distributions

Les histogrammes générés pour chaque distribution et chaque taille d'échantillon n permettent d'observer les phénomènes suivants :

- **Uniforme entière** : Pour $n = 10$, l'histogramme est très irrégulier en raison du faible nombre d'échantillons. Lorsque n augmente (100, 1000, 10000), l'histogramme se rapproche d'une distribution uniforme discrète sur $[20, 40]$, avec une fréquence similaire pour chaque valeur entière. ““
- **Uniforme réelle** : Pour $n = 10$, les valeurs sont dispersées. Avec $n = 10000$, l'histogramme devient presque plat sur l'intervalle $[2.0, 3.0]$, confirmant la nature uniforme de la distribution.
- **Normale** : Pour $n = 10$, la distribution semble aléatoire. À partir de $n = 1000$, la forme en cloche caractéristique de la distribution normale (centrée sur $\mu = 0$ avec un écart-type $\sigma = 0.5$) devient visible. Avec $n = 10000$, la courbe est presque parfaite.
- **Exponentielle** : Pour $n = 10$, les valeurs sont très variables. Avec $n = 10000$, l'histogramme montre une décroissance exponentielle, typique de cette distribution (paramètre d'échelle $\text{scale} = 4$).
- **Poisson** : Pour $n = 10$, les valeurs sont peu représentatives. Avec $n = 10000$, l'histogramme montre une distribution discrète centrée autour de $\lambda = 1$, avec une décroissance rapide des fréquences pour les valeurs éloignées de la moyenne. ““

5.2 Interprétation

- **Convergence** : Plus n est grand, plus l'histogramme se rapproche de la distribution théorique. Cela illustre la **loi des grands nombres**, qui stipule que la moyenne empirique converge vers l'espérance théorique lorsque la taille de l'échantillon tend vers l'infini.

- **Biais d'échantillonnage** : Pour $n = 10$, les histogrammes sont très sensibles aux fluctuations aléatoires. Cela montre l'importance de choisir une taille d'échantillon suffisante pour obtenir des résultats fiables.
- **Visualisation** : Les histogrammes permettent de valider visuellement que les générateurs aléatoires de `numpy` produisent bien les distributions attendues.

5.3 Exemple de figures

Les figures générées (ex : `histUniforme.png`, `histNormale.png`) montrent clairement l'évolution des histogrammes en fonction de n . Voici un exemple de ce à quoi ressemble l'histogramme pour la distribution normale avec $n = 10000$:



FIGURE 1 – Histogramme de la distribution normale pour $n = 10000$.

6 Conclusion

Ce projet démontre comment modéliser et visualiser des distributions statistiques en Python à l'aide de `numpy` et `matplotlib`. Les résultats confirment que les générateurs aléatoires de `numpy` sont fiables et que les histogrammes

convergent vers les distributions théoriques lorsque la taille de l'échantillon augmente.

6.1 Pistes d'amélioration

- **Tests statistiques** : Ajouter des tests (ex : test de Kolmogorov-Smirnov) pour valider quantitativement que les échantillons suivent bien les distributions théoriques.
- **Distributions supplémentaires** : Étendre le code pour inclure d'autres distributions (ex : binomiale, géométrique, etc.).
- **Optimisation** : Utiliser des méthodes de traçage plus efficaces pour les très grands échantillons (ex : `plt.hist` avec `density=True` pour comparer directement avec la densité théorique).
- **Interface utilisateur** : Créer une interface graphique (ex : avec `Tkinter` ou `Streamlit`) pour permettre à l'utilisateur de choisir interactivement les paramètres des distributions.